Pipeline Gráfico de OpenGL

Del Modelo Analítico Continuo al Framebuffer en Silicio

Dr. J. Antonio Aviña Méndez

ANTECEDENTES

¿Qué es OpenGL? El Paradigma de la Máquina de Estados

- **El Panel de Control:** OpenGL no es Orientado a Objetos. Funciona como un gigantesco panel de interruptores globales conectados a la GPU.
- **Estado Persistente:** Las configuraciones permanecen en el hardware hasta que se modifiquen explícitamente.
- **Despacho Atómico:** La instrucción glDrawArrays no recibe parámetros; procesa la orden usando el estado activo en ese preciso instante.

ANTECEDENTES • VISUAL

Entendiendo los Estados en OpenGL



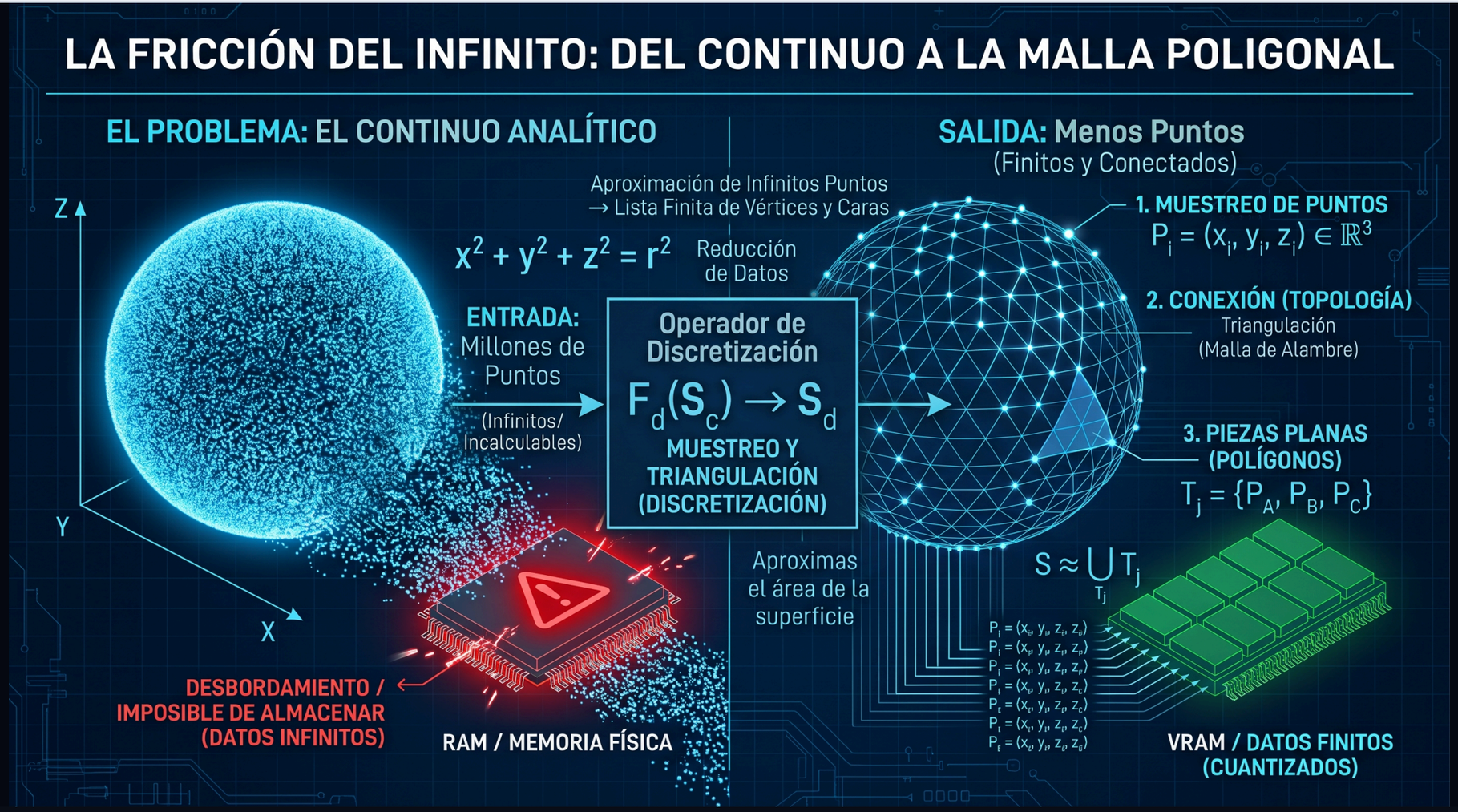
ANTECEDENTES

La Discretización: La Fricción del Infinito

- **El Problema Físico:** Las ecuaciones algebraicas del continuo $\mathbb{R}^3$ poseen infinitos puntos. Es imposible almacenarlos en memorias RAM/VRAM finitas.
- **Cuantización:** Traducir curvas matemáticas incalculables a mallas de datos estructurados procesables por el silicio.
- **El Triángulo como Primitivo Universal:** Figura geométrica mínima que garantiza coplanaridad estricta, acelerando cálculos masivos.

ANTECEDENTES • VISUAL

El Choque con el Hardware



ANTECEDENTES

El Espacio Canónico NDC

- **Estandarización de GPU:** El hardware ignora la resolución de tu pantalla. Todo vértice debe proyectarse en un sistema abstracto universal.
- **El Cubo Estándar:** Espacio estricto donde $\Omega_{NDC} = [-1, 1]^3$.
- **Descarte Automático:** Cualquier coordenada matemática por fuera del rango $[-1, 1]$ es descartada.

OpenGL 2D Normalized Device Coordinates (NDC)

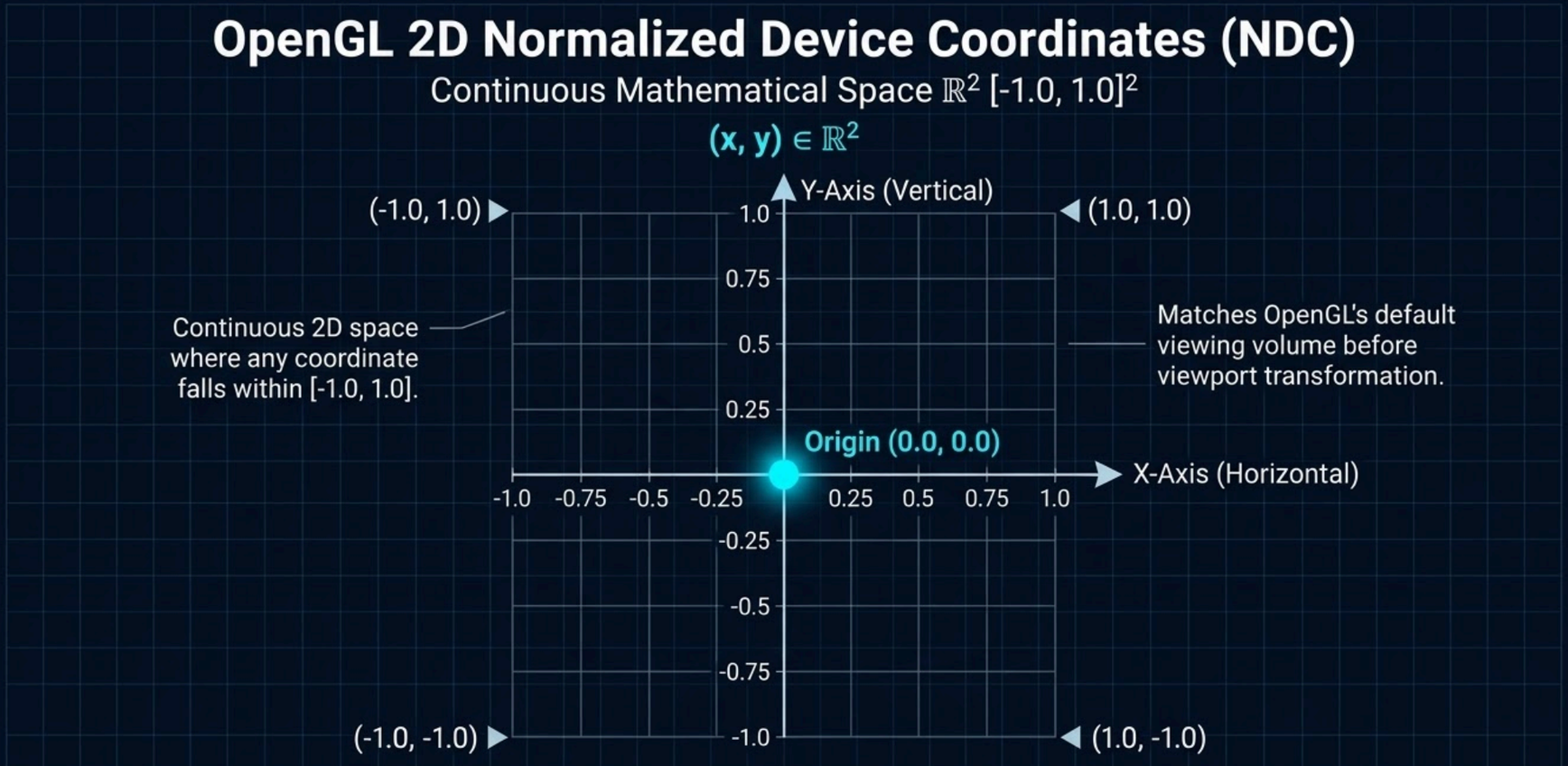


Fig 1. OpenGL 2D Normalized Device Coordinate (NDC) Space: A continuous mathematical Cartesian plane ranging from -1.0 to +1.0 on both X and Y axes, with (0.0, 0.0) at the center.

ANTECEDENTES

La Transformación de Viewport

- **Puente al Hardware:** Mapeo afín del cubo continuo hacia píxeles discretos de la pantalla $\mathbb{Z}^2$.
- **Mapeo de Coordenadas:** Escala y traslada de $[-1, 1]$ a las dimensiones físicas de la ventana.
- **Profundidad:** Inversión del eje Y y mapeo del eje Z hacia $[0, 1]$ para el Z-Buffer.

NDC vs. Espacio Discreto de Pantalla

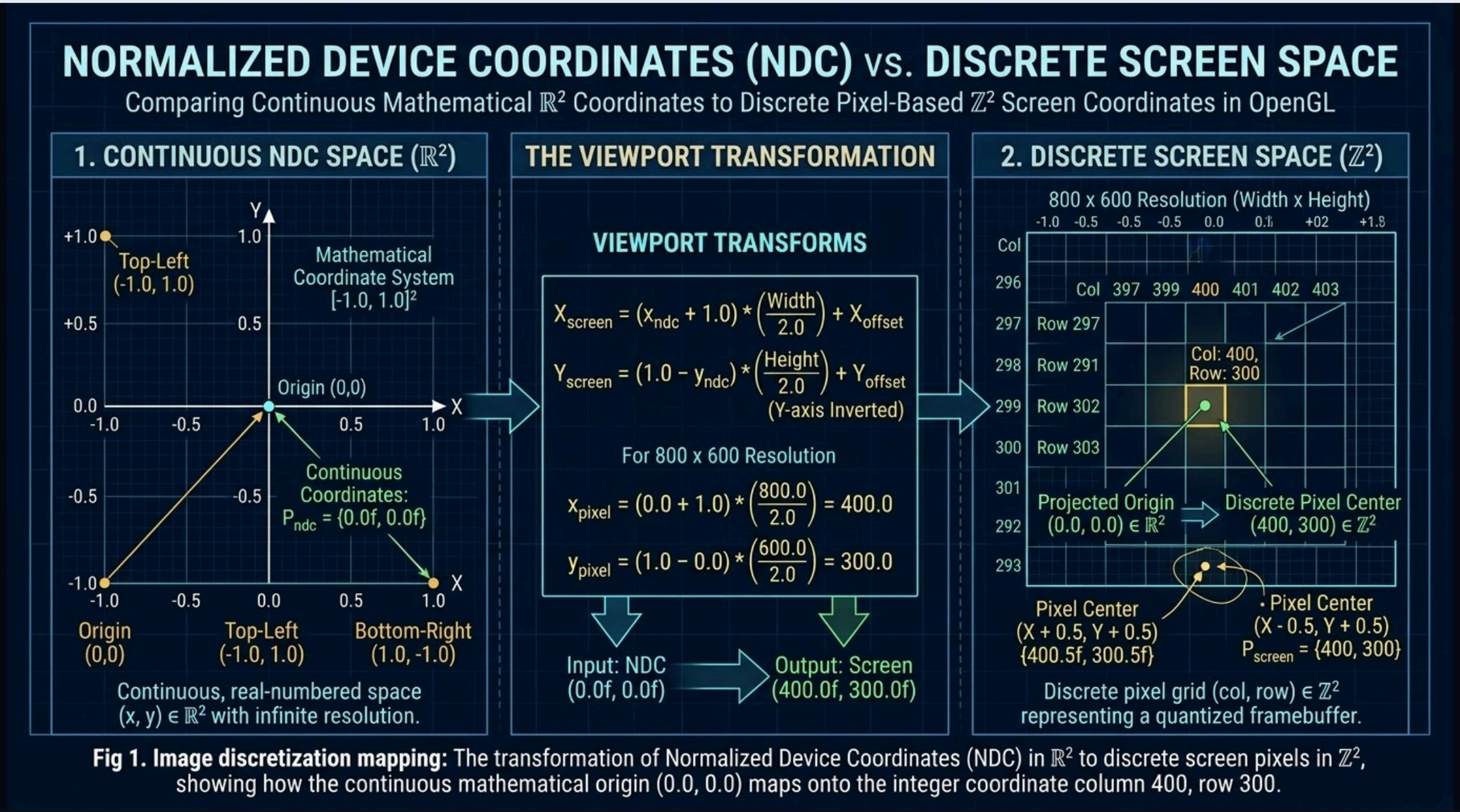


Fig 1. Image discretization mapping: The transformation of Normalized Device Coordinates (NDC) in $\mathbb{R}^2$ to discrete screen pixels in $\mathbb{Z}^2$, showing how the continuous mathematical origin (0.0, 0.0) maps onto the integer coordinate column 400, row 300.

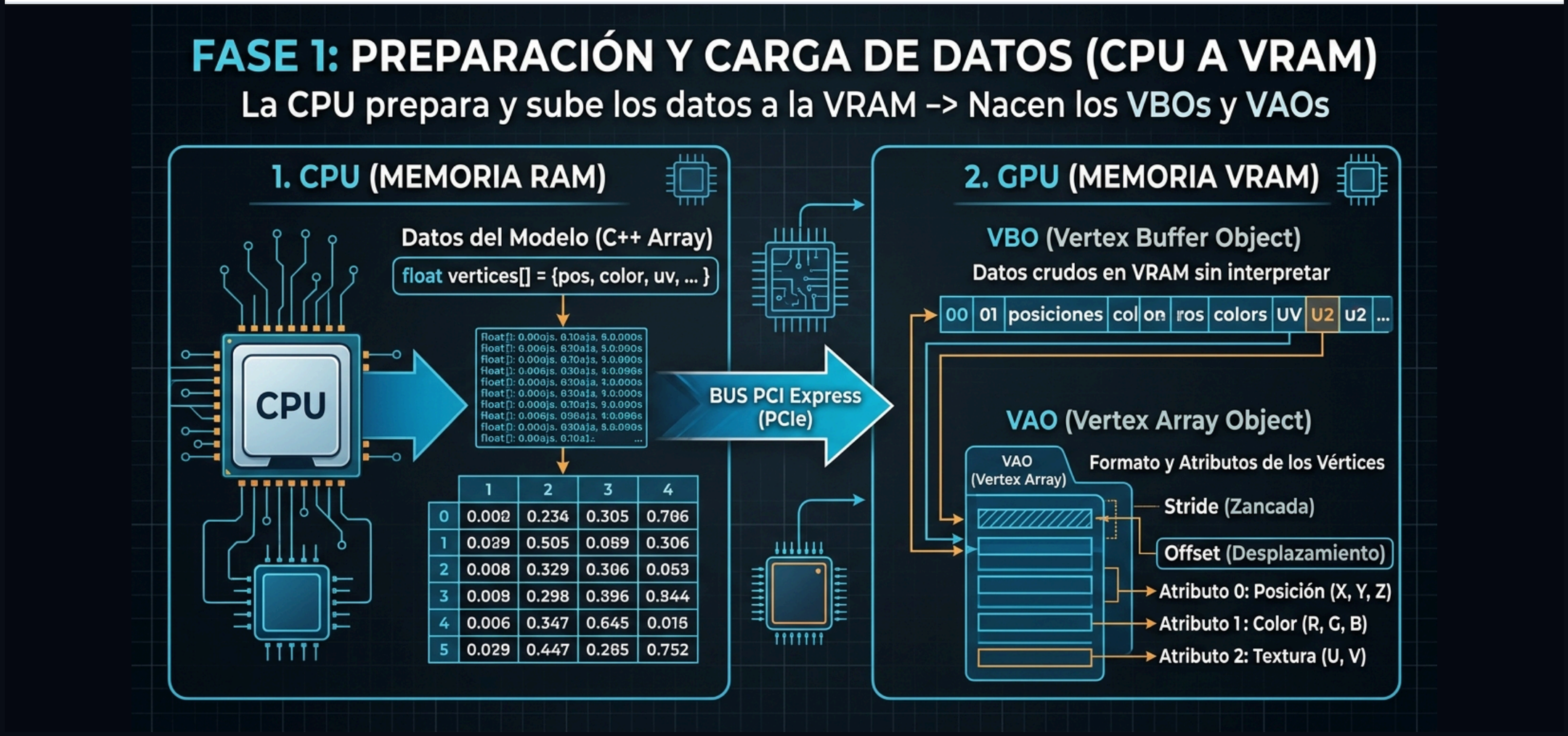
FASE 1

Transferencia a VRAM y Arquitectura VBO/VAO

- **Bus PCIe:** Envío de memoria en bloque hacia la GPU para máxima eficiencia.
- **VBO (Vertex Buffer Object):** Fragmento de memoria contigua en la VRAM reservado para datos crudos.
- **VAO (Vertex Array Object):** Objeto de estado que recuerda cómo interpretar el VBO (Stride y Offsets).

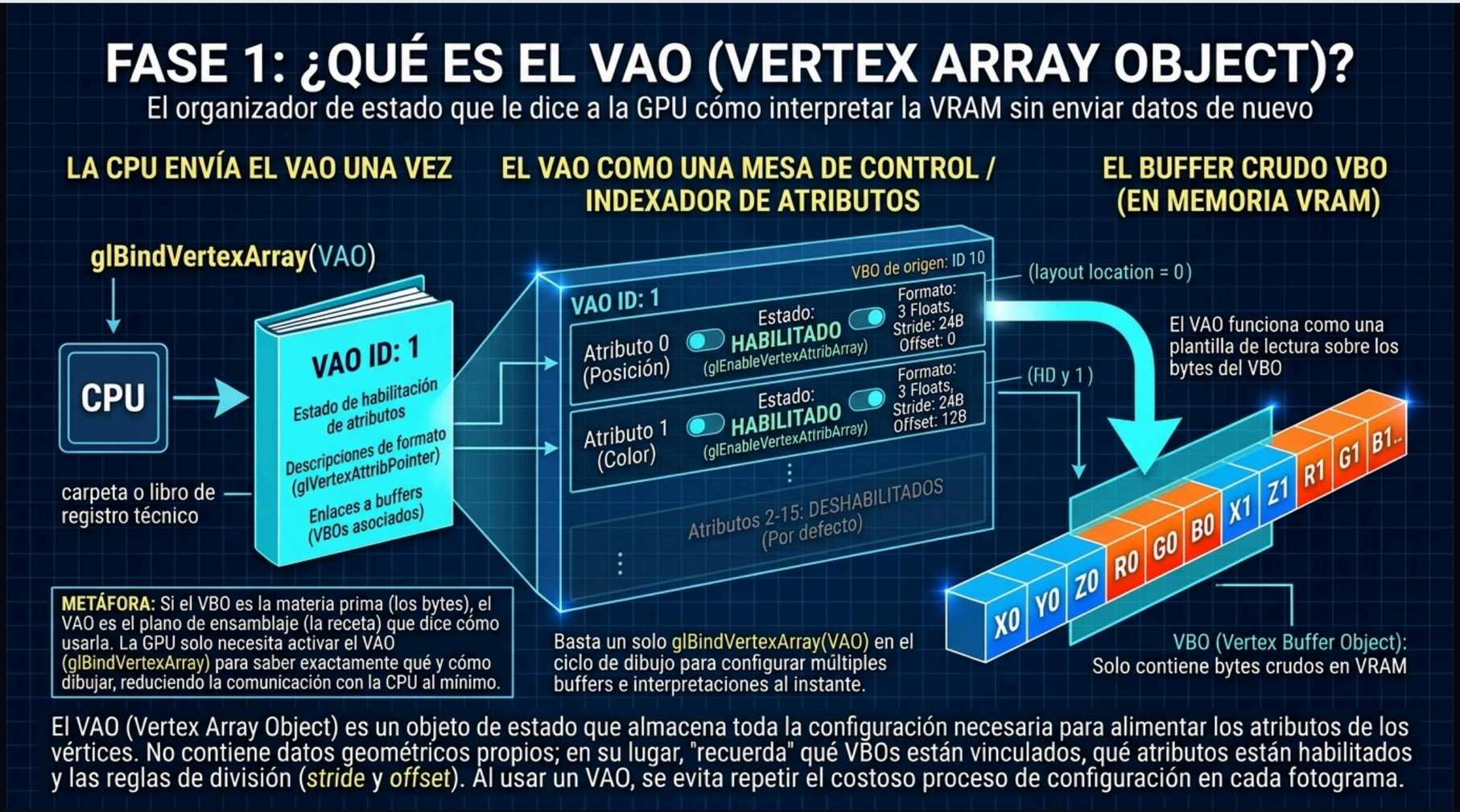
FASE 1 • VISUAL

VBO y Carga Masiva



FASE 1 • VISUAL

¿Qué es el VAO?



El VAO (Vertex Array Object) es un objeto de estado que almacena toda la configuración necesaria para alimentar los atributos de los vértices. No contiene datos geométricos propios; en su lugar, "recuerda" qué VBOs están vinculados, qué atributos están habilitados y las reglas de división (*stride* y *offset*). Al usar un VAO, se evita repetir el costoso proceso de configuración en cada fotograma.

FASE 1 • CÓDIGO (C++)

Configuración

```
GLuint VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);

glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

FASE 2

Procesamiento Paralelo y Vertex Shader

- **SIMT Paralelo:** Millones de hilos calculando simultáneamente sin dependencias entre vértices.
- **Cadena MVP:** Proyección matemática obligatoria:

$$\mathbf{P}_{\mathbf{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{P}_{\mathbf{local}}$$

- **Destino:** Escribir coordenadas proyectadas homogéneas en gl_Position.

FASE 2 • VISUAL

Procesamiento SIMT Paralelo



El **Vertex Shader** es una etapa obligatoria y programable del *pipeline* que se ejecuta en paralelo en miles de núcleos de la GPU, una vez por cada vértice. Su tarea principal es tomar los datos en bruto de cada punto y proyectarlos en el espacio de recorte 3D (*Clip Space*) multiplicando su posición por la matriz MVP. El resultado se guarda en la variable del sistema `gl_Position`, preparando la geometría para la rasterización.

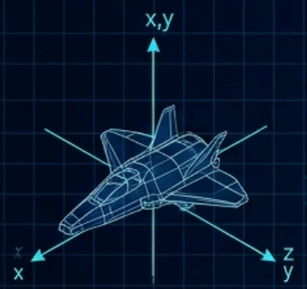
FASE 2 • VISUAL

La Cadena MVP

LAS TRES MATRICES DE TRANSFORMACIÓN (MVP)

De la geometría local de un modelo al espacio proyectivo 2D de la pantalla

EL OBJETO EN SU ESPACIO LOCAL

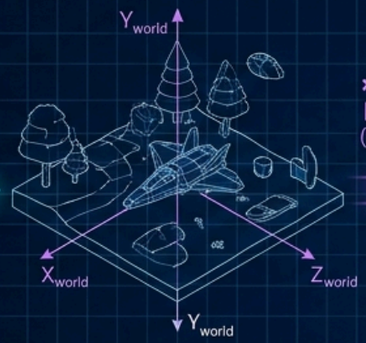


- **Espacio de Objeto / Local:** Las coordenadas del modelo están definidas en relación a su propio centro.

× MATRIZ DE MODELO (Model Matrix)

- **Matriz de Modelo:** Escala, rota y traslada el objeto para colocarlo en su posición dentro del mundo 3D.

EL OBJETO COLOCADO EN EL MUNDO

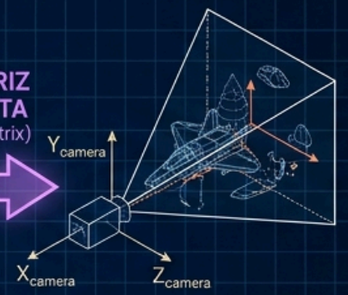


- **Espacio del Mundo (World Space):** Todos los objetos del escenario comparten un sistema de referencia único.

× MATRIZ DE VISTA (View Matrix)

- **Matriz de Vista:** Desplaza y rota todo el mundo en sentido inverso para simular la posición y orientación de la cámara (el observador).

EL MUNDO VISTO DESDE LA CÁMARA



- **Espacio de Cámara / Vista (View Space):** Las coordenadas se recalculan poniendo el lente de la cámara en el origen (0,0,0) mirando hacia -Z.

× MATRIZ DE PROYECCIÓN (Projection Matrix)

- **Matriz de Proyección:** Aplica la perspectiva (los objetos lejanos se ven pequeños) o la vista ortográfica, y define el volumen visible (Frustum) de recorte.

EL VOLUMEN DE RECORTE (CLIP SPACE)



- **Espacio de Recorte (Clip Space):** La geometría está en coordenadas homogéneas de 4D (x,y,z,w), lista para el recortionados listos para ser recortadón final a 2D en pantalla (NDC).

El Vertex Shader toma cada vértice en su espacio local y lo multiplica en orden por la cadena de transformación: V_final = Proyección × Vista × Modelo × V_local. Estas tres matrices (MVP) son fundamentales en los gráficos 3D modernos para colocar objetos en el mundo (Modelo), posicionar al observador/cámara (Vista) y distorsionar los datos densisador/cámara (Vista) y distorsionar los datos con perspectiva para la visualización en un plano 2D (Proyección).

FASE 2 • CÓDIGO (GLSL)

Vertex Shader

```
#version 330 core
layout (location = 0) in vec3 aPos;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

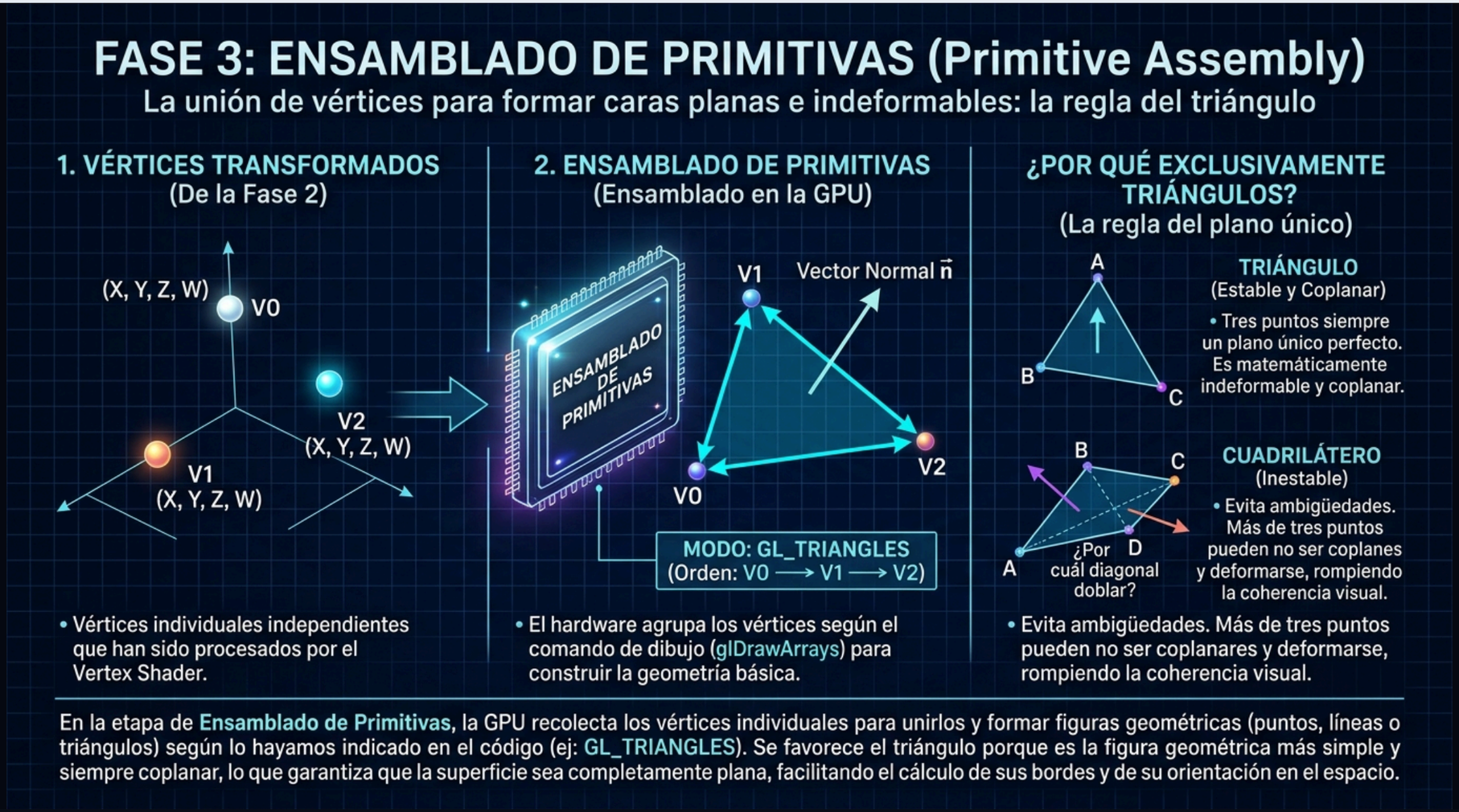
FASE 3

Ensamblado de Primitivas y Face Culling

- **Ensamblado:** Conexión de vértices procesados en triángulos matemáticos.
- **Frustum Clipping:** Segmentación de geometría que cruza los límites de visión.
- **Face Culling:** Descarte automático del 50% de polígonos mediante el análisis del devanado CCW (Counter-Clockwise).

FASE 3 • VISUAL

Primitive Assembly y Clipping



FASE 3 • CÓDIGO (C++)

Optimización Geométrica

```
// Habilitar descarte de caras ocultas
glEnable(GL_CULL_FACE);

// Definir frente como antihorario
glFrontFace(GL_CCW);

// Descartar la cara trasera
glCullFace(GL_BACK);

glDrawArrays(GL_TRIANGLES, 0, 36);
```

FASE 4

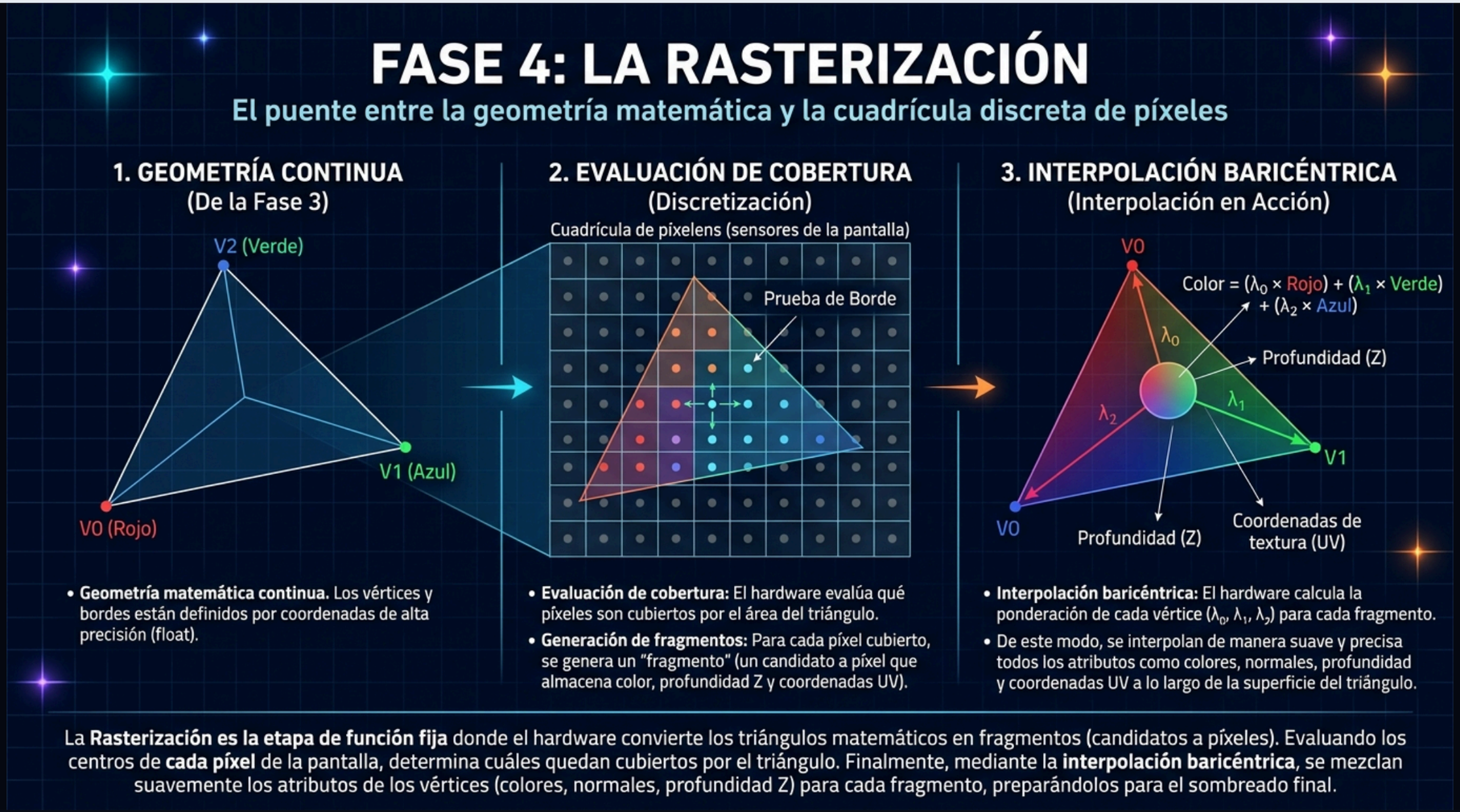
Rasterización

- **Muestreo Subpíxel:** Evaluación de polígonos sobre la cuadrícula del monitor generadora de "Fragmentos".
- **Interpolación Baricéntrica:** Fusión continua de atributos matemáticos hacia el interior del triángulo:

$$\mathbf{p} = \lambda_0 \mathbf{v}_0 + \lambda_1 \mathbf{v}_1 + \lambda_2 \mathbf{v}_2$$

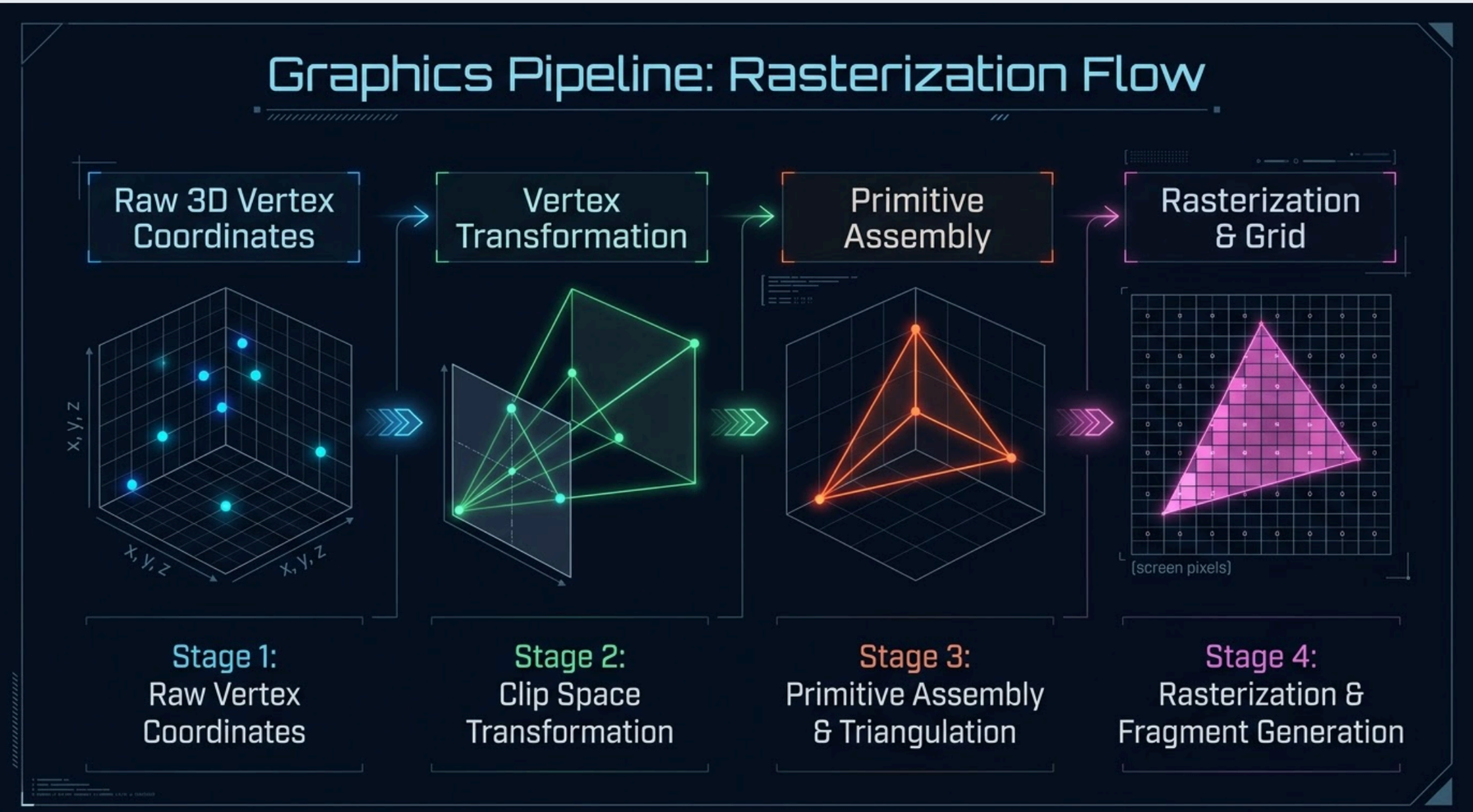
FASE 4 • VISUAL

La Rasterización



FASE 4 • VISUAL

Flujo del Rasterizador



FASE 4 • VISUAL

Etapas Físicas

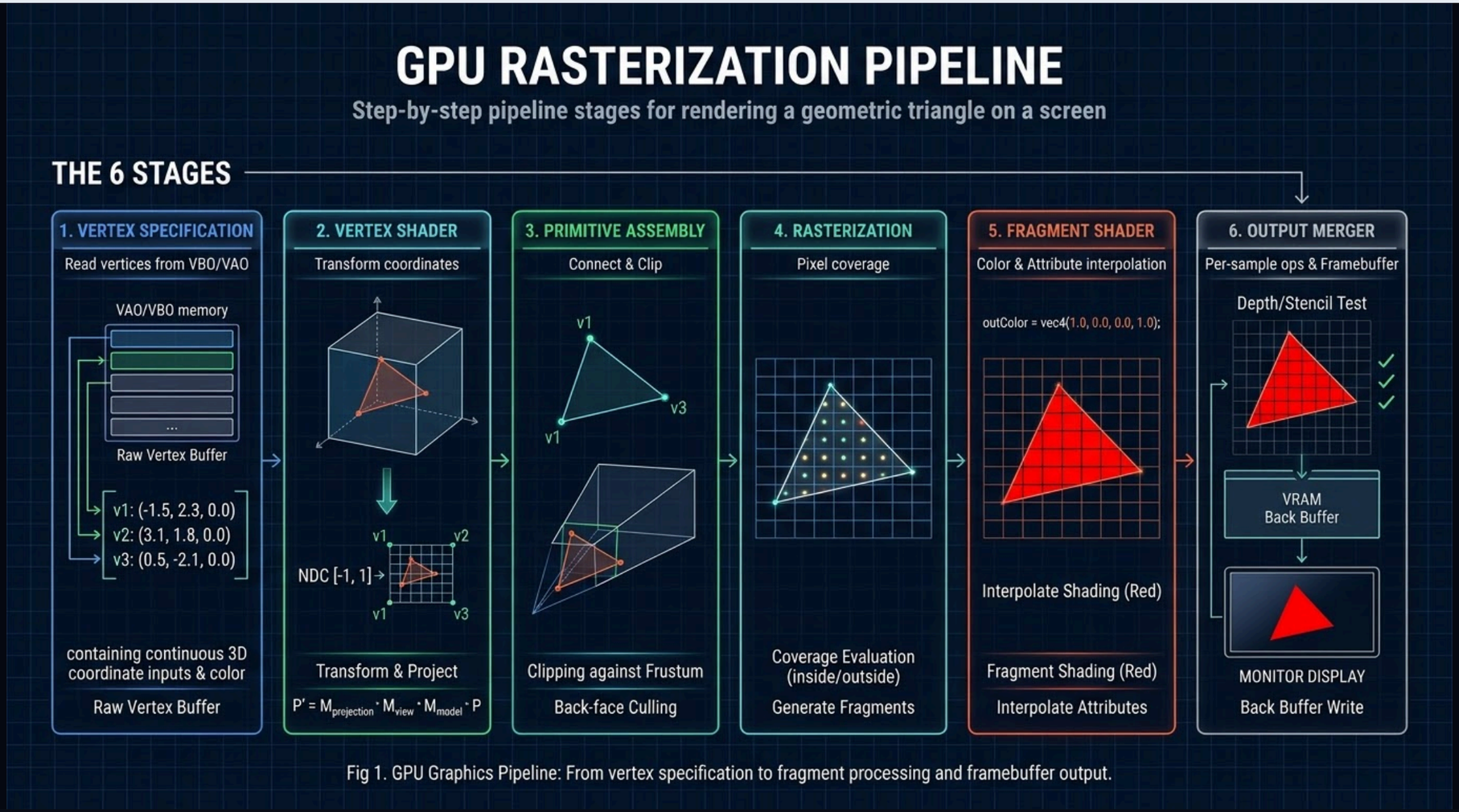


Fig 1. GPU Graphics Pipeline: From vertex specification to fragment processing and framebuffer output.

FASE 4 • CÓDIGO (C++)

Control del Rasterizador

```
// Definir matriz de discretización (Mapeo a Píxeles)
glViewport(0, 0, 1920, 1080);

// Forzar rasterizado de malla de alambre
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);

// Retornar a relleno sólido
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

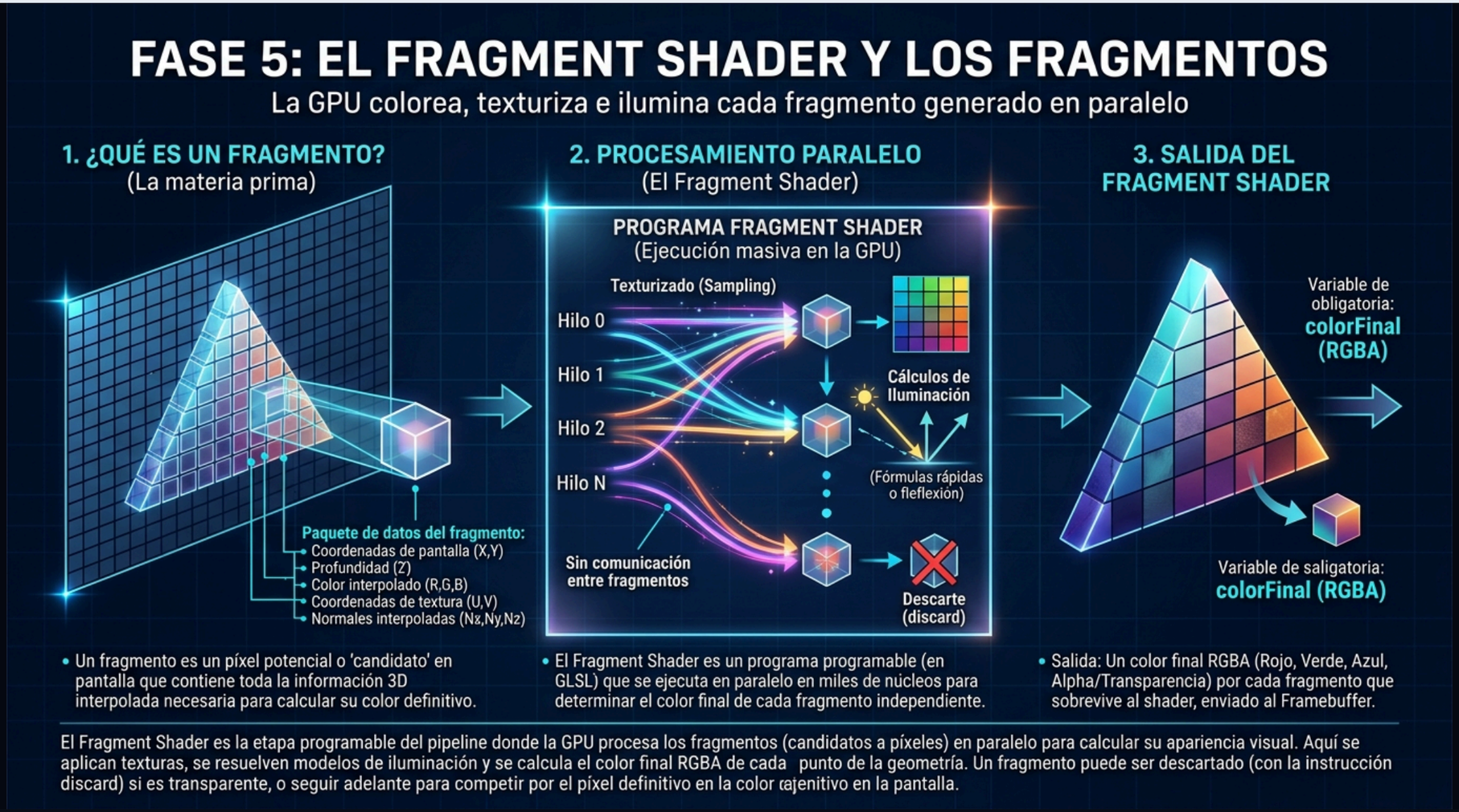
FASE 5

Sombreado de Fragmentos

- **Fragment Shader:** Algoritmo de coloración ejecutado por cada fragmento generado en el rasterizador.
- **Texturizado Continuo:** Lectura y muestreo paramétrico sobre texturas.
- **Descarte:** Uso del comando discard para abortar el procesamiento de píxeles transparentes de forma nativa.

FASE 5 • VISUAL

El Fragment Shader



FASE 5 • CÓDIGO (GLSL)

Fragment Shader

```
#version 330 core
out vec4 FragColor;
in vec2 TexCoord;
uniform sampler2D texture1;

void main() {
    vec4 texColor = texture(texture1, TexCoord);
    if(texColor.a < 0.1)
        discard;

    FragColor = texColor;
}
```

FASE 6

Pruebas y Salida al Framebuffer

- **Z-Buffer (Depth Test):** Condición de oclusión física evaluada como:

$$Z_{\text{frag}} < Z_{\text{buffer}}$$

- **Blending:** Mezcla de opacidades sobre canales RGBA.
- **Doble Búfer:** Prevención del "Tearing" construyendo en un buffer oculto y volcando el bloque completo al monitor sincrónicamente.

FASE 6 • VISUAL

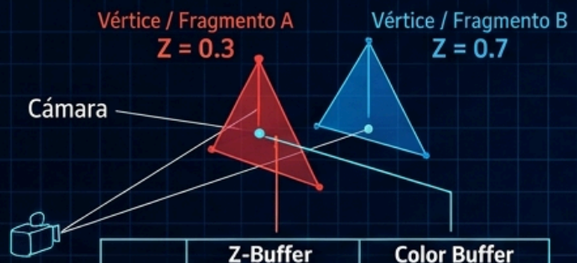
Depth Test y Salida

FASE 6: DEPTH TEST Y SALIDA AL FRAMEBUFFER

Se descartan los elementos ocultos y se escribe en la pantalla

1. PRUEBAS POR MUESTRA (Depth Test)

- Dos fragmentos compiten por el mismo píxel de pantalla



	Z-Buffer (Depth Buffer)	Color Buffer
rojo	Z = 0.3	1.0
velte	Z = 0.3	1.0
azul	Z = 0.7	0.3

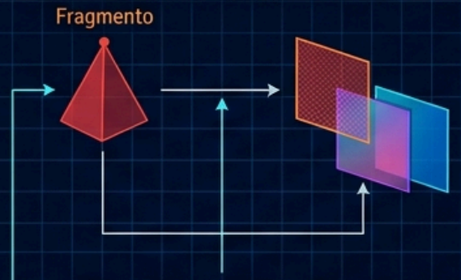
$Z_{fragmento} < Z_buffer \longrightarrow$ PASA

$Z_{fragmento} > Z_buffer \longrightarrow$ SE DESCARTA

- **Prueba de Profundidad (Depth Test):** La GPU compara la profundidad Z de cada fragmento con el valor guardado en el Depth Buffer. Si el fragmento está más cerca, se escribe y se actualiza el Z-Buffer; si está tapado, se descarta para evitar que los objetos traseros se encimen sobre los delanteros.

2. OPERACIONES DE MEZCLA (Blending) Y COMPOSICIÓN

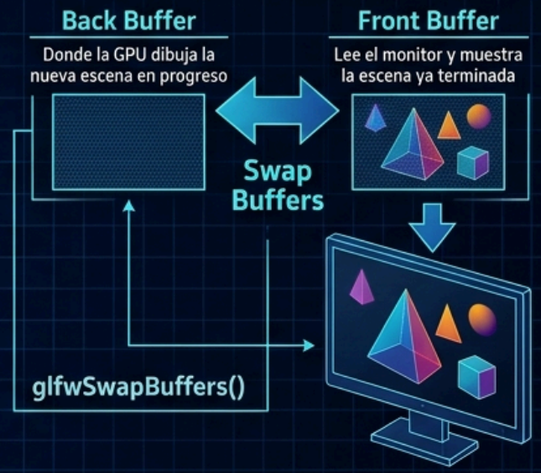
- **Pruebas y mezcla final: pasaàai:** Se aplican filtros adicionales y operaciones de mezcla (Blending) si tiene transparencia (Blending), o simplemente sobrescribe el color anterior en la VRAM.



Pero la **prueba de plantilla (Stencil Test)** y la **prueba de recorte (Scissor Test)** como filtros previos al guardado final.

- **Pruebas y mezcla final:** Se aplican filtros adicionales y operaciones de mezcla (Blending) para transparencia antes de escribir los colores definitivos.

3. DOBLE BÚFER (Double Buffering) Y PANTALLA



- **El Framebuffer:** El Back Buffer recibe los píxeles aprobados para la siguiente imagen. Una vez listo, se realiza el intercambio (Swap Buffers) con el Front Buffer, que es el que el monitor lee a 60 cuadros por segundo, evitando el parpadeo y la ruptura visual (tearing).

La Fase 6 es la última etapa del pipeline donde se decide **qué fragmentos se convierten en píxeles visibles de pantalla.** A través de la **Prueba de Profundidad** (Depth Test), la GPU utiliza el Z-Buffer para descartar de forma automática los objetos que quedan tapados por otros. Los píxeles ganadores se guardan en el **Back Buffer del Framebuffer** y, mediante el **doble búfer** (double buffering con **glfwSwapBuffers**), se envían al monitor para mostrar el cuadro de manera fluida y sin parpadeos.

FASE 6 • CÓDIGO (C++)

Control del Framebuffer

```
// Activar la prueba de profundidad en hardware
glEnable(GL_DEPTH_TEST);

// Limpiar buffers en cada frame
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

// Intercambiar buffer oculto al monitor visible
glfwSwapBuffers(window);
```

Síntesis del Pipeline Gráfico

1. **Transferencia:** CPU a VRAM mediante arreglos (VBO/VAO).
2. **Vertex Shader:** Transformación de coordenadas locales hacia recorte (MVP).
3. **Ensamblado:** Creación de primitivas y descarte de espaldas (Culling/Clipping).
4. **Rasterización:** Discretización bidimensional en pixeles.
5. **Fragment Shader:** Sombreado masivo y muestreo de texturas.
6. **Framebuffer:** Oclusiones Z y vaciado al monitor.